Filière Apprentis 5ème Année

Spécialité EIE

Compte rendu TP1

Projet robotique : Résolution d'un labyrinthe avec Dijkstra et A*

Réalisé par:

- DOUBLET Jeremy
- SALL Mohamadou

Encadré par: Mr QUANG - TRUNG Luu

1. Introduction

Ce TP a pour objectif de comprendre et d'implémenter des algorithmes de planification de trajectoire dans un environnement discret. Ce qui consiste à modéliser un labyrinthe sous forme de grille 2D, définir un point de départ et un objectif, puis par la suite d' appliquer deux algorithmes de recherche: Dijkstra et A*.

Ces algorithmes permettent de trouver le plus court chemin entre deux points tout en évitant les obstacles. On utilisera ensuite ne visualisation graphique pour observer le fonctionnement des algorithmes, notamment les cases explorées pendant la recherche ainsi que le chemin final trouvé.

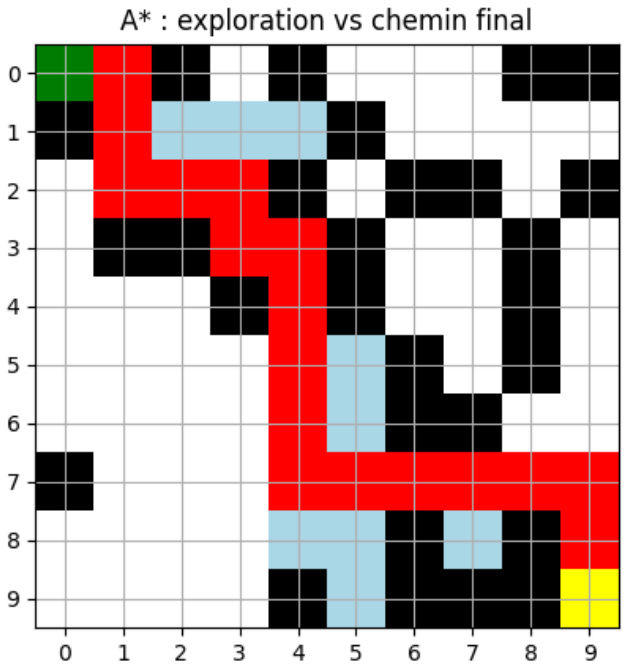
2. Modélisation du labyrinthe

Le labyrinthe est représenté par une grille 2D. Chaque cellule de la grille peut être :Libre → le robot peut passer ; Obstacle → le robot ne peut pas passer
Dans notre implémentation :

- `0` représente une cellule libre
- `1` représente un obstacle

Le robot possède va donc posseder deux positions : une position de départ start une position cible goal.

voici un exemple de labyrinthe généré



La classe principale du programme est :

```
class Maze:

   def __init__(self, width, height, start, goal):

       self.width = width

       self.height = height

       self.start = start

       self.goal = goal

       self.grid = [[0 for _ in range(width)] for _ in range(height)]
```

Ceci est notre classe principale qui va contenir les elements suivants : la grille, la position de départ, la position d'arrivée , les fonctions de résolution.

3. Génération du labyrinthe

Les obstacles du labyrinthe sont générés **de manière aléatoire** avec une probabilité donnée. Dans notre programme, cela est réalisé avec la fonction :

```
   def generate_random_obstacles(self, obstacle_prob=0.3):

       for y in range(self.height):

           for x in range(self.width):

               if (x, y) != self.start and (x, y) != self.goal:

                   if random.random() < obstacle_prob:

                       self.grid[y][x] = 1
```

Ca va prendre comme paramètre une probabilité par exemple 0.3. Cela signifie que chaque cellule de la grille a 30 % de chance de devenir un obstacle.

Ce qui va permettre de créer différents labyrinthes et de simuler un environnement plus réaliste, contenant à la fois des zones libres où le robot peut se déplacer et des zones bloquées par des obstacles. Le point de départ et l'objectif ne peuvent jamais être des obstacles.

4. Déplacements dans la grille

Les voisinages de deplacement du robot sont limités : droite ; gauche ; haut et vers le bas.

Ce qui est representé par :

```
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
```

Ca permettra d'assurer que la position reste dans la grille et que la case n'est pas un obstacle

5. Algorithme de Dijkstra

- Principe

L'algorithme de Dijkstra permet de trouver le plus court chemin entre une source et les autres sommets d'un graphe.

Il fonctionne comme suit : explore progressivement les nœuds puis sélectionne toujours celui avec la distance minimale et mets à jour les distances de ses voisins.

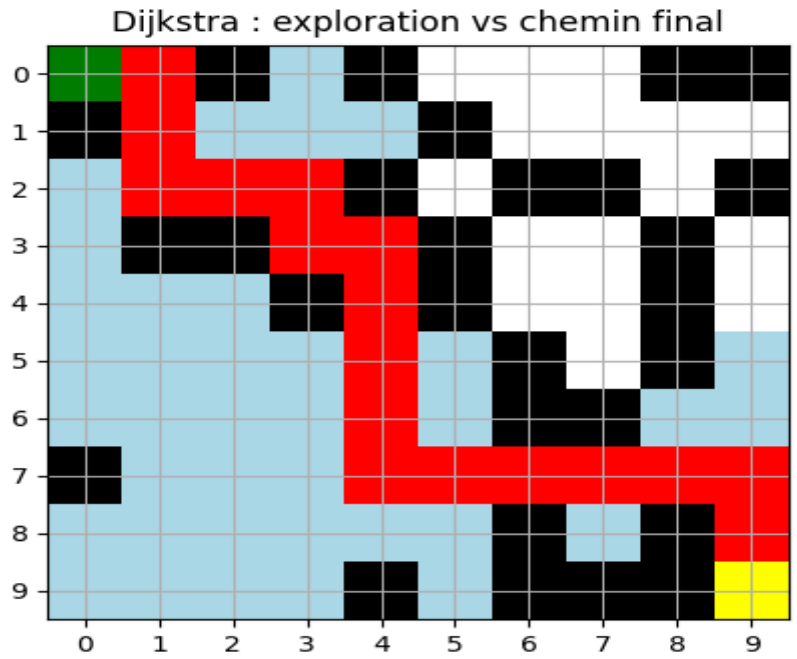
Dans notre TP: chaque déplacement coûte 1 , on cherche à atteindre la cellule objectif

Implémentation :

```
    def solve_dijkstra(self):
```

Une file de priorité (heapq) est utilisée pour sélectionner la cellule avec la plus petite distance.

Voici le résultat obtenu avec l'algo dijkstra :



6. Algorithme A*

- Principe

L'algorithme **A*** lui est une amélioration de Dijkstra utilisant une heuristique pour guider la recherche.

La fonction d'évaluation est :

$$f(n) = g(n) + h(n)$$

avec :

- g(n) : coût réel depuis le départ
- h(n) : estimation du coût restant

On a egalment utilisé la distance de Manhattan qui est donné par

$$h(x, y) = |x - xg| + |y - yg|$$

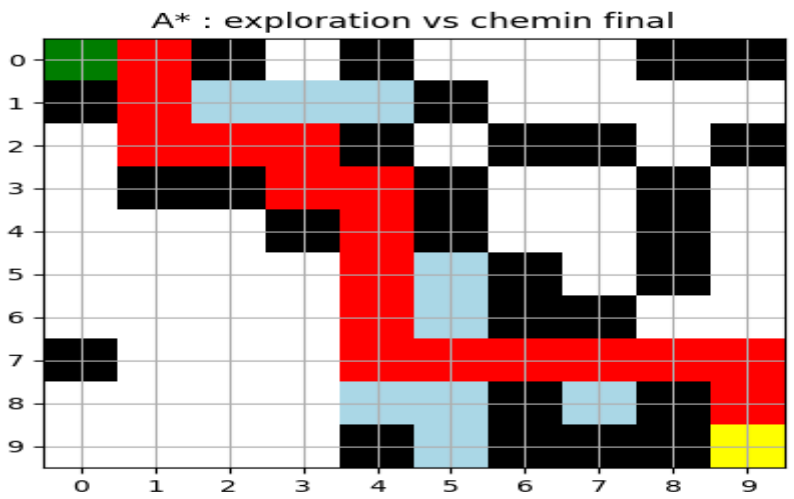
Cette distance est adaptée aux grilles où seuls les mouvements horizontaux et verticaux sont autorisés.

```
def heuristic(self, x, y):
```

et le

```
def solve_astar self
```

Voici le résultat obtenu avec l'algo A*:



7. Reconstruction du chemin

Une fois l'objectif atteint, on reconstruit le chemin en remontant les parents des nœuds :

```
def reconstruct_path self  came_from
```

```
        current = self.goal

        path = [current]

        while current in came_from:

            current = came_from[current]

            path.append(current)

        path.reverse()

        return path
```

Le chemin est ensuite inversé pour obtenir :

start → ... → goal

8. Test du programme

Le labyrinthe utilisé :

```
maze = Maze(10, 10, (0, 0), (9,9))
maze.generate_random_obstacles(0.3)
```

Les parametres utilisés ici sont : grille 10 × 10 ; départ (0,0) ; objectif (9,9) ; 30 % d'obstacles

Les deux algorithmes sont exécutés :

```
# A*
path_a, explored_a = maze.solve_astar()
maze.plot(path_a, explored_a, "A* : exploration vs chemin final")

print("A* - cases explorées :", len(explored_a))
print("A* - longueur chemin :", len(path_a) if path_a else "Aucun chemin")

# Dijkstra
path_d, explored_d = maze.solve_dijkstra()
maze.plot(path_d, explored_d, "Dijkstra : exploration vs chemin final")

print("Dijkstra - cases explorées :", len(explored_d))
print("Dijkstra - longueur chemin :", len(path_d) if path_d else "Aucun chemin")
```

Les résultats affichés sont : le nombre de cases explorées , la longueur du chemin

9. Analyse des résultats et comparaison des algo

Dijkstra

- explore beaucoup de cellules
- ne possède aucune information sur la position de l'objectif
- garantit le chemin optimal

A*

- utilise une heuristique
- explore moins de cellules
- converge plus rapidement vers l'objectif

En pratique, **A*** est souvent plus efficace que Dijkstra pour les problèmes de navigation.

10. Conclusion

Dans ce TP, nous avons donc modélisé un labyrinthe sous forme de grille 2D afin d'étudier le problème de la recherche de chemin dans un environnement discret. Pour celà 2 algos on était utilisés : l'algorithme de Dijkstra et l'algorithme A*, pour avoir donc le plus court chemin entre une position de départ et un objectif, tout en évitant les obstacles présents dans la grille. Grâce à la visualisation graphique, nous avons pu observer les zones explorées par les algorithmes ainsi que le chemin optimal trouvé.

Aussi on a pu faire une petite comparaison à la fin ; l'algorithme de Dijkstra explore généralement une grande partie de l'espace, car il ne possède pas d'information sur la direction de l'objectif. À l'inverse, l'algorithme A* utilise une heuristique qui lui permet de guider la recherche vers la cible, ce qui le rend généralement plus efficace. Ce type d'algorithme est largement utilisé dans plusieurs domaines tels que la robotique mobile, les jeux vidéo, la navigation autonome et la planification de trajectoire.

ANNEXE : lien github pour notre code

https://github.com/jeremydoubletge2-lang/Projet_IA_Maze